



Geodesic Forests

Meghana Madhyastha
Department of Computer Science,
Johns Hopkins University

Gongkai Li
Department of Applied Mathematics
and Statistics, Johns Hopkins
University

Veronika Strnadová-Neeley
Department of Computer Science,
Montana State University

James Browne
Department of Computer Science,
Johns Hopkins University

Joshua T. Vogelstein
Department of Biomedical
Engineering, Johns Hopkins
University

Randal Burns
Department of Computer Science,
Johns Hopkins University

Carey E. Priebe
Department of Applied Mathematics
and Statistics, Johns Hopkins
University

ABSTRACT

Together with the curse of dimensionality, nonlinear dependencies in large data sets persist as major challenges in data mining tasks. A reliable way to accurately preserve nonlinear structure is to compute geodesic distances between data points. Manifold learning methods, such as Isomap, aim to preserve geodesic distances in a Riemannian manifold. However, as manifold learning algorithms operate on the ambient dimensionality of the data, the essential step of geodesic distance computation is sensitive to high-dimensional noise. Therefore, a direct application of these algorithms to high-dimensional, noisy data often yields unsatisfactory results and does not accurately capture nonlinear structure.

We propose an unsupervised random forest approach called geodesic forests (GF) to geodesic distance estimation in linear and nonlinear manifolds with noise. GF operates on low-dimensional sparse linear combinations of features, rather than the full observed dimensionality. To choose the optimal split in a computationally efficient fashion, we developed Fast-BIC, a fast Bayesian Information Criterion statistic for Gaussian mixture models. We additionally propose geodesic precision and geodesic recall as novel evaluation metrics that quantify how well the geodesic distances of a latent manifold are preserved. Empirical results on simulated and real data demonstrate that GF is robust to high-dimensional noise, whereas other methods, such as Isomap, UMAP, and FLANN, quickly deteriorate in such settings. Notably, GF is able to estimate geodesic distances better than other approaches on a real connectome dataset.

CCS CONCEPTS

• **Computing methodologies** → **Dimensionality reduction and manifold learning**; • **Theory of computation** → Random projections and metric embeddings; • **Mathematics of computing** → *Probabilistic algorithms*.

KEYWORDS

noisy data, random forest, unsupervised, manifold learning

ACM Reference Format:

Meghana Madhyastha, Gongkai Li, Veronika Strnadová-Neeley, James Browne, Joshua T. Vogelstein, Randal Burns, and Carey E. Priebe. 2020. Geodesic Forests. In *26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '20)*, August 23–27, 2020, Virtual Event, USA. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3394486.3403094>

1 INTRODUCTION

Nearest neighbor algorithms, which sort all points according to their distances to one another, are considered among the top 10 most important data mining algorithms of all time [51] and have strong theoretical guarantees for both classification and regression [43]. Decision trees (such as CART and C4.5) can also reasonably be thought of as algorithms for organizing data in a hierarchical fashion; these are among the top ten algorithms as well [51]. Moreover, decision trees underlie both random forests [7] and gradient boosted trees [22], which are the two leading algorithms for machine learning on tabular data today [9–11]. There is a rich literature on approximate nearest neighbor algorithms (see Aumäijller et al. [2] for benchmark comparisons), which are used extensively in big data systems.

However, operating on the exact nearest neighbors, (or trying to approximate them), is not always desirable. For example, consider a simple supervised learning setting. Given a data corpus, $\{(x_n, y_n)\}_{n=1}^N$, learn a decision rule that predicts y for a given new data point x and achieves a small error with high probability. A canonical approach is kernel regression [37]. A kernel machine's prediction is a weighted linear combination of predictions that the neighbors of x would make, specifically, $\hat{y} = \frac{1}{N} \sum_{n=1}^N y_n \times \kappa(x, x_n)$, for some suitably chosen kernel κ (for example, a radial basis function or k -nearest neighbors kernel). Such approaches enjoy strong

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

KDD '20, August 23–27, 2020, Virtual Event, USA

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-7998-4/20/08...\$15.00

<https://doi.org/10.1145/3394486.3403094>

theoretical guarantees [33]. Now, further assume that the x 's are noisy measurements of some true, but unobserved $\tilde{x}$'s. Such an assumption, called “measurement error modeling” [24], could reasonably be argued to be much more accurate than assuming the x 's are noise-free measurements [25]. Under this measurement error assumption, an approach that achieves smaller error given the same sample size would be a kernel regression function on the noise-free measurements, $\hat{y} = \frac{1}{N} \sum_{n=1}^N y_n \times \kappa(\tilde{x}, \tilde{x}_n)$. Unfortunately, because the $\tilde{x}$'s are not observed, such an approach is not available. This modeling framework highlights the need for learning latent structure which sample points are close to one another on an underlying latent structure (such as a manifold), for subsequent inference. Moreover, even though the above describes a supervised learning task, performance may improve when learning the structure of only the features x , while not even considering the labels y , as we will demonstrate in this paper.

Learning latent structure is even more important when the dimensionality p is larger than the sample size N . In such a setting, an intermediate representation is required to avoid (1) numerical stability issues with matrix operations, and (2) the “curse of dimensionality” for statistical operations. This intermediate representation can be explicit (as in manifold learning) or implicit (as in kernel machines). When N is large, even approximation algorithms are required to compute various quantities whose exact solution requires $O(N^2)$ or $O(N^3)$ space and/or time.

Geodesic distance is the shortest path between two points in a Riemannian manifold, and geodesic learning is the process of estimating geodesic distances between pairs of points in a data corpus. Given this definition, it would be reasonable to consider using manifold learning algorithms to first learn the latent structure of the data, and then estimate distances in the estimated latent structure space. In fact, the first step in many manifold learning algorithms is to estimate the geodesic distance between all pairs of points [27]. Nonetheless, this is not a common practice. Rather, others have used manifold learning algorithms (such as random projection) to estimate the observed, noisy, and often high-dimensional ambient distances. Yet, several disciplines in computer science and machine learning have developed strategies to partially address these challenges.

In computer science, space-partitioning trees are used extensively for quite diverse applications; most relevant to this work are efforts to build trees in support of efficient geometric queries [46]. Space partitioning trees use binary and recursive splits with hyperplanes [14]. These tree structures are usually optimized to learn relative proximities of the observed, noisy measurements, rather than the latent noise-free, and potentially lower dimensional, measurements. Partitioning trees are closely related to decision trees in statistics and machine learning [8]. Extensions to decision trees are the de facto standard for classification and regression tasks (even in this age of deep learning), including random forests [7] and gradient boosting trees [23]. However, decision trees are almost exclusively concerned with supervised, rather than unsupervised learning.

Analogies between decision trees and kernel learning have been studied for decades. Breiman [6] showed that in a sense, random forests are equivalent to a kernel acting on the true margin. This

realization recently gained traction in the machine learning literature [3, 17, 38, 39]. In manifold learning, many spectral variants start by estimating all pairwise geodesic distances [36]. These approaches operate on the observed, high-dimensional data and suffer serious performance loss in the face of additional noise dimensions. Although each of the above works is closely related to geodesic learning, none of them explicitly define and estimate geodesic distances.

We propose *Geodesic Forest* (GF) to achieve near linear space and time complexity, while approximating the true latent geodesic distances. Unlike the previously described methods, GF does not need to compute geodesic distances between all pairs of points. Instead, GF examines local structure by recursively clustering data in sparse linear subspaces, building on the recently proposed randomer forest algorithm for supervised learning [48]. The randomer forest approach allows GF to separate meaningful structure in the data from the noise dimensions. We also introduce a spitting criteria, Fast-BIC, which efficiently and exactly computes the Bayesian Information Criterion statistic for an approximate Gaussian mixture model in one dimension.

This manuscript also contributes a method for evaluating geodesic learning algorithms. Most existing work on manifold learning that explicitly estimates geodesic distances, does not explicitly evaluate the geodesics. Rather, such techniques embed the data into a low-dimensional space and then visualize the results. This approach is limited in a number of ways: (1) it is qualitative; (2) when the structure is higher dimensional it may not be revealed by the first few dimensions; and (3) it relies on an embedding, which introduces additional computational and statistical complications. Sometimes the embedded data are used for subsequent inference tasks, such as classification, which can be quantitatively evaluated. These approaches are only able to evaluate performance of a manifold learning algorithm composed with a particular subsequent inferential method, but not the manifold or geodesic learning algorithms directly.

Therefore, we introduce geodesic precision and recall. In contrast to precision and recall, geodesic precision and recall compare the set of nearest neighbors, as estimated by the geodesic learning algorithm, with the set of true nearest neighbors on the latent manifolds. If a geodesic learning algorithm does poorly on this metric, estimating manifolds from these geodesics has no hope to perform well on subsequent tasks. Indeed, functions of geodesic precision can provide tight bounds on subsequent classification accuracy [18].

GF finds neighbors in the latent low-dimensional space amid additional noise dimensions more effectively than other approaches. Moreover it can do this in a variety of linear and nonlinear settings, with different dimensional submanifolds. In addition, we demonstrate that GF is more effective than other approaches in distinguishing cell types in a real, recently released, connectome dataset.

2 RELATED WORK

Nonlinear manifold learning approaches based on Isomap [45] are designed to preserve geodesic distances. First, they estimate geodesic distances in the original manifold. This is done by constructing

a k -nearest neighbor or ϵ -neighborhood graph in which the observations (data points) correspond to nodes, and pairwise Euclidean distances between these points correspond to the weights on the edges. Second, the all-pairs shortest paths of the nodes in the graph are computed. Third, the points are embedded in a lower dimensional space that ideally preserves these distances. This approach is hampered by the first step, which operates in the original high-dimensional ambient space, because Euclidean distances often fail to provide good estimates of distances on the manifold. Moreover, computing all pairwise distances among n datapoints takes $O(n^2)$ space and time and all pairwise shortest paths can require $O(n^3)$, both of which can be cost prohibitive for large sample sizes.

Isomap is one of the few manifold learning algorithms that has theoretical guarantees for correctly estimating the manifold [42]. However, Isomap stores all point-to-point graph distances, which incurs space and time complexity quadratic in the sample size. Laplacian eigenmaps [4], Hessian eigenmaps [19], and diffusion maps [12] try to address the limitations of Isomap by learning embeddings that preserve other properties of the data, such as “diffusion distance”, while maintaining some theoretical guarantees. However, in the case of many noisy dimensions, these approaches fail to construct accurate nearest-neighbor graphs on the latent manifold, because they base the k -neighbor graph on Euclidean distances in the observed space.

UMAP is an algorithm for dimensionality reduction that reduces high-dimensional data to a low dimension using a fuzzy simplicial set representation of the input data points [31]. Like other nearest-neighbor based algorithms, UMAP constructs an undirected, weighted k -nearest neighbor graph from the input data, then embeds data points in a low-dimensional space using a force-directed layout algorithm. The number of neighbors used to construct the graph determines the local manifold structure that is to be preserved in the low-dimensional layout. In the force-directed layout, attractive forces between close vertices are iteratively balanced with repulsive forces between vertices that are far apart in the graph until convergence. UMAP builds upon the t-Distributed Stochastic Neighbor Embedding (t-SNE) algorithm, which attempts to preserve original interpoint distances in a lower dimensional space [29]. The Kullback-Leibler divergence between the distribution of neighbor distances in the higher and lower dimensional spaces is used to determine the optimal mapping of points into the lower-dimensional space. t-SNE is primarily used to visualize high-dimensional data [30], and cannot be used with non-metric distances. UMAP produces similar embeddings to t-SNE in two or three dimensions, but scales better in run-time across a wide range of embedding dimensions [31].

Approximate nearest neighbors algorithms, such as FLANN [34], approximate nearest-neighbors in high-dimensional data sets by building binary space-partitioning trees, such as K -d trees. These algorithms are designed to estimate the distances in the observed high-dimensional space. When the true manifold is low-dimensional and the data are high-dimensional, the additional noise dimensions are problematic for any of these algorithms. These approaches achieve near linear space and time complexity.

This work is inspired by, and closely related to, random projection trees for manifold learning [15] and vector quantization [16]. The main differences between our approach and theirs is (1) that

they use random splits, rather than optimizing the splits; and (2) they use a single tree, whereas GF uses a forest of many trees. Nonetheless, their theoretical analysis motivates the geodesic precision metric that we establish for quantifying the performance of geodesic learning.

Finally, most closely related to our method are unsupervised random forest methods. The construction of manifold forests as described by Criminisi et al. [13] is an unsupervised random forest approach. To find the best split at each internal node, an information gain measure based on the differential entropy of a multivariate Gaussian is optimized, leading to a computationally expensive procedure. In addition, to our knowledge, this approach has not been thoroughly evaluated on highly noisy data sets. The most popular unsupervised random forest method is included in Cutler’s Random-Forest R package [40]. It proceeds by generating a synthetic copy of the data by randomly permuting each feature independently of the others and then attempts to classify the real versus the synthetic dataset. We will show below that this approach can miss simple latent structures.

3 GEODESIC FORESTS

A random forest is an ensemble of decision trees in which each tree is created from bootstrapped samples of the training dataset; that is, each tree is built from a random subset of training data. Each tree $\{h(\mathbf{x}, \theta_t)\}$, $t \in \{1, 2, \dots, T\}$ has parameters θ_t that characterize the tree structure, and can be learned from the dataset. Given a set of trees, and a new point x , each tree casts a unit vote for its predictions given the input $\mathbf{x}$. Typically, random forests are used in supervised machine learning tasks, specifically classification and regression. There have been a few papers reporting on unsupervised random forests for certain tasks [41].

Our geodesic forest algorithm is based on the original Random Forest algorithm [7] with a few key distinctions. First, GF uses a new splitting criteria, Fast-BIC, that efficiently and exactly computes an approximate Bayesian Information Criterion for a Gaussian Mixture model in one dimension. Second, we use the term *randomer* to label our technique, as our splitting methods are based on random sparse linear combinations of features to strengthen each tree, as originally proposed by Breiman [7], and later studied further by Tomita et al. [47, 48] Third, we correctly implement a previously proposed method for generating proximity matrices from random forests. In one of the most widely used implementations of Random Forest [28], the aggregated normalized proximity matrices of F Random Forests with T trees each is not stochastically equivalent to the aggregated normalized proximity matrices of T Random Forests with F trees each. Our implementation does not suffer from this bug. Furthermore, it is computationally more efficient than previous implementations. These three changes enable GF to achieve state-of-the-art performance on both simulated and real data.

3.1 GF Algorithm Overview

Here, we describe the geodesic forest algorithm in detail. Given an input data set $x = \{x_1, \dots, x_N\}$, where $x_n \in \mathbb{R}^p$, GF builds T decision trees, each from a random sample of size $m < N$. In each tree, GF recursively splits a parent node into its two child nodes until some termination specification is met. At each node,

GF generates d features to search over. Each feature is evaluated based on the splitting criteria described in Section 3.3, and the feature with the best score is selected to split the data points into two daughter nodes. Algorithm 1 describes the procedure used to build unsupervised decision trees (all algorithms are relegated to the appendix). To evaluate the forest, a proximity matrix is then generated by computing the fraction of the trees in which every pair of elements reside in the same leaf node (Section 3.4).

3.2 Node-Wise Feature Generation

Unlike Breiman’s original random forest algorithm, GF does not choose split points in the original feature space. Instead, we follow the random projection framework of Tomita et al. [47, 48]. For p -dimensional input data, we sample a $p \times d$ matrix A distributed as f_A , where f_A is the projection distribution and d is the dimensionality of the projected space. We chose to use the f_A that Tomita et al. empirically found to produce the best performance in the supervised setting: A is generated by randomly sampling from $\{-1, +1\}$ $\lambda p d$ times, then distributing these values uniformly at random in A [48]. The λ parameter is used to control the sparsity of A , and is set to $\frac{1}{20}$, again following the convention of Tomita et al. Using the randomly sampled $p \times d$ matrix A , the data associated with the given node, X' , which is a subsample of the original data, is transformed into a d -dimensional feature space, where each of the d new features is now a sparse linear combination of the p original features. In other words, each row of $\tilde{X} = A^T X'$ represents a projection of the data into a one-dimensional space that is a sparse linear combination of the original feature space. Each of the d rows $\tilde{X}[i, :], i \in \{1, 2, \dots, d\}$ is then inspected for the best split point. The optimal split point and splitting dimension are chosen according to which point/dimension pair minimizes the splitting criteria described in the following section.

3.3 Splitting Criteria

We present three alternatives for choosing the best split among the one-dimensional set of points associated with a GF tree node.

1-Fast, Exact, Univariate, Two-Means Splitting. This method seeks to find the cutpoint that minimizes the one-dimensional 2-means objective. This splitting criteria [15] is:

$$\min_s \sum_{n=1}^s (x_n - \hat{\mu}_1)^2 + \sum_{n=s+1}^N (x_n - \hat{\mu}_2)^2. \quad (1)$$

The goal is to find the split that minimizes the sum of the intra-cluster variance on the projected dimension. Typically, k-means problems are solved via Ward’s or Hardigan’s algorithms [26, 50]. Because k-means is NP-hard, in general, these algorithms lack strong theoretical guarantees [1]. However, in one-dimension, for two-means, there is an exact solution that is much faster and simpler. This is available because each decision tree always operates on one-dimensional marginals which represent the probability specific to a single dimension. First, sort the data points. Then, consider splitting between all sequential pairs of points; that is, letting $x_{(s)}$ denote the s^{th} smallest sample, consider splitting between $x_{(s)}$ and $x_{(s+1)}$ for all $s < N$. The samples to the left of the split point form one cluster,

and those to the right form the other cluster. Estimate the means for each of the clusters using the maximum likelihood estimate (MLE).

An immediate limitation of this approach is that it fails to consider feature-wise variance, which can lead to undesirable properties. For example, if any feature has zero variance, it will always achieve the minimum possible score. Although one can rescale each feature independently, doing so can cause problems in unsupervised learning problems when the relative scale of features is important, and the details of how to rescale introduce an undesirable algorithm parameter to tune.

2-Gaussian Mixture Model Splitting with Mclust-BIC. For each feature, we fit the data to a two-component Gaussian mixture model (GMM). An expectation-maximization (EM) is used to jointly estimate all the parameters and latent variables [21]. The latent variables, $\{z_{n,j}\}$, denote the probability that sample n is in cluster j . Letting N be the number of observations and J be the number of Gaussian clusters (in this case, $J = 2$), and introducing notation $x = (x_1, \dots, x_N)$, and $z = \{z_{1,1}, z_{1,2}, \dots, z_{N,J}\}$, the complete likelihood (including the latent variables) is

$$P(x, z; \mu, \sigma, \pi) = \prod_{n=1}^N \prod_{j=1}^J \{\pi_j \mathcal{N}(x_n; \mu_j, \sigma_j^2)\}^{z_{n,j}}. \quad (2)$$

Each feature is evaluated using the Bayesian Information Criterion (BIC). BIC is based on the log likelihood of the model given the data, with a regularization term penalizing complex models with many parameters. Concretely, letting $\hat{L}_M$ denote the maximum log likelihood function of a particular model M , $\hat{L}_M = p(x; \hat{\theta}_M)$, where $\hat{\theta}_M$ are the parameters that maximize the likelihood function for model M , and x is the observed data. Letting N be the sample size (number of data points) and d_M be the number of parameters estimated by the model, then the BIC score can be defined as follows:

$$BIC(M) = -2 \ln(\hat{L}_M) + \ln(N) d_M. \quad (3)$$

The feature that maximizes the BIC score for a two-component GMM is selected for splitting at each node. The split occurs at the midpoint where the two Gaussians are equally likely. Because this approach is standard in the literature, we do not provide pseudocode. Note that the EM approximates the actual log likelihood and is only guaranteed to find a local maximum, not the global maximum. Thus, it is sensitive to initialization. Moreover, the EM algorithm suffers from poor convergence properties in certain settings [32].

2-GMM Splitting with Fast-BIC. Our new Fast-BIC method combines the speed of two-means with the model flexibility of Mclust-BIC. As in two-means, for each feature, we sort all the data and try all possible splits. For each split, we assign all points below the split to one Gaussian and all points above the split to the other Gaussian. We estimate the prior, means and variances for both clusters using the MLE. For $j = 1$, they are defined by

$$\hat{\mu}_1 = \frac{1}{s} \sum_{n \leq s} x_n, \quad \hat{\sigma}_1 = \frac{1}{s} \sum_{n \leq s} \|x_n - \hat{\mu}_1\|^2, \quad \hat{\pi}_1 = \frac{s}{N},$$

and similarly for $j = 2$. Under the above assumption, $z_{n,j}$ is an indicator that data point x_n is in cluster j . In other words, rather than the soft clustering of GMM, Fast-BIC performs a hard clustering, as in two-means. Thus, if x_n is in cluster j , then $z_{n,j} = 1$ and

$z_{n,j'} = 0$ for all $j \neq j'$. Given this approximation, the likelihood can be obtained by summing over the z 's

$$P(x; \mu, \sigma, \pi) = \sum_z \prod_{n=1}^N \prod_{j=1}^J \{\pi_j \mathcal{N}(x_n; \mu_j, \sigma_j^2)\}^{z_{n,j}}. \quad (4)$$

Noting that $z_{(n \in [0, s], k=0)} = z_{(n \in [s+1, N], k=1)} = 1$ and $z_{n,j} = 0$ otherwise, Equation (4) can be simplified to

$$P(x; \mu, \sigma, \pi) = \prod_{n=1}^s \pi_1 \mathcal{N}(x_n; \hat{\mu}_1, \hat{\sigma}_1^2) \prod_{n=s+1}^N \pi_2 \mathcal{N}(x_n; \hat{\mu}_2, \hat{\sigma}_2^2).$$

Plugging in the MLE for all the parameters, the maximum log likelihood function $\hat{L} = \log P(x; \hat{\mu}, \hat{\sigma}, \hat{\pi})$ is

$$\hat{L} = \sum_{n=1}^s [\log \hat{\pi}_1 + \log \mathcal{N}(x_n; \hat{\mu}_1, \hat{\sigma}_1^2)] + \sum_{n=s+1}^N [\log \hat{\pi}_2 + \log \mathcal{N}(x_n; \hat{\mu}_2, \hat{\sigma}_2^2)]. \quad (5)$$

Substituting into Equation 5 and simplifying, we get the following expression for the log likelihood for any given s :

$$-2\hat{L}_s = s \log 2\hat{\pi}_1 \hat{\sigma}_1^2 + (N-s) \log 2\hat{\pi}_2 \hat{\sigma}_2^2 - s \log \hat{\mu}_1 - (N-s) \log \hat{\mu}_2, \quad (6)$$

in which we have dropped terms that are not functions of the parameters. We further test for the single variance case ($\sigma_1 = \sigma_2$) and use the BIC formula to determine the best case. Fast-BIC chooses the dimension and split-point that maximizes $\hat{L}_s$. Pseudocode for this approach is provided in Algorithm 3.

Fast-BIC is guaranteed to obtain the global maximum likelihood estimator, whereas the Mclust-BIC is liable to find only a local maximum. Moreover, Fast-BIC is substantially faster. This Fast-BIC procedure is, to our knowledge, novel, and of independent interest.

3.4 Proximity Matrix Construction

One can build a similarity matrix from any decision tree by asserting that similarity between two points x_n and x_j is related to some “tree distance” between the pair of points in a given tree. Although this is the case for both supervised and unsupervised decision trees, to our knowledge this approach has not yet been explored for unsupervised trees. When using a forest, it is natural to average the similarity matrices to obtain the forest’s estimate of similarity. A simple tree distance to use is the 0–1 loss on whether a pair of points is in the same leaf node. This approach to computing similarities has previously been studied in several supervised random forest papers, connecting random forests to kernel learning [3, 6, 17, 38, 39]. However, the connection between these similarities and geodesic distances has not yet been established.

More concretely, the proximity matrix S for input data $D \in \mathbb{R}^{n \times d}$ is estimated using the geodesic forest by simply counting the fraction of times that a pair of points occurs in the same leaf node in the forest. Thus, $S(i, j) = S_{ij} = \frac{L_{ij}}{T_{ij}}$, where $L(i, j)$ is the number of occurrences of points i and j in the same leaf node, and T_{ij} is the number of trees in which both point i and point j were included in the bootstrap sample that was used to build the tree. We use both the in-bag and out-of-bag samples to estimate the proximity.

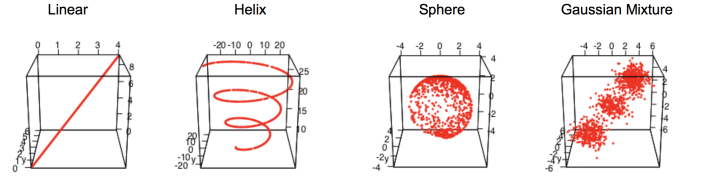


Figure 1: Synthetic datasets for all experiments. In each case, there are 1000 points in 3 signal dimensions, as shown.

3.5 Geodesic Precision and Recall

We present the novel *geodesic precision* and *geodesic recall* evaluation metrics to determine how well a method estimates geodesic distances. With these metrics, we can compare the geodesic distances for points on the latent manifold to the estimated distances produced by GF. Geodesic precision and recall differ from “classical” precision and recall by virtue of defining the neighbors based on the true latent low-dimensional manifold, rather than the observed (typically higher-dimensional) space. The typical definitions of precision and recall are defined relative to a query. The *relevant samples* are those that are “correct” in the sense that they are truly relevant to the query, where as the *retrieved samples* are those that are returned by the query. For geodesic learning, given a data point x , a data corpus $\mathcal{D}_N = \{x_1, \dots, x_N\}$, and a query size k , the relevant samples are the k samples from $\mathcal{D}_N$ that are nearest to x based on the true (but unknown) geodesic distance. In other words, *relevant neighbors* are those points that are nearest on the latent, noise-free, manifold, rather than the observed, typically higher dimensional space. Given a geodesic learner, the *retrieved neighbors* are the k samples that the learner reports are nearest. Letting $\cap$ denote set intersection, and $|\cdot|$ denote the cardinality of the set, geodesic precision and recall are defined as:

$$\text{geodesic precision} = \frac{|\{\text{manifold neighbors}\} \cap \{\text{retrieved neighbors}\}|}{|\{\text{retrieved neighbors}\}|},$$

$$\text{geodesic recall} = \frac{|\{\text{manifold neighbors}\} \cap \{\text{retrieved neighbors}\}|}{|\{\text{manifold neighbors}\}|}.$$

To compute the geodesic precision and recall for a given learner on a given dataset, we average the geodesic precision and recall over each sample point. Higher precision and lower recall indicate a better estimation of geodesic distances.

We consider two distinct cases: a continuous geodesic in which there is a finite geodesic distance between all pairs of points and a discrete geodesic in which there are clusters of points that are not connected at all to other points. In the latter case (such as a union of spheres), we denote all the points within a given connected component as its neighbors, and all points outside its connected component as not neighbors. In the disconnected setting, geodesic precision and recall are identical to one another.

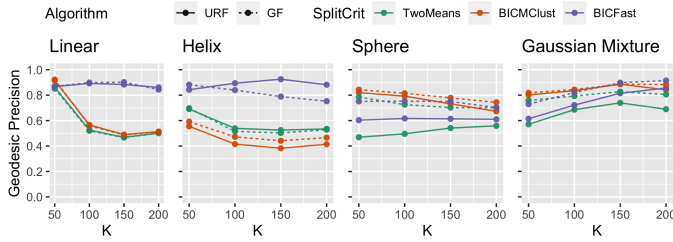


Figure 2: Geodesic precision curves for the three different splitting criteria using both axis-aligned splits (URF; solid lines) and sparse oblique splits (GF; dashed lines). In each case there are 1000 points and 3 signal dimensions (no noise dimensions). In general, GF with Fast-BIC performs the best or nearly so.

4 NUMERICAL RESULTS

4.1 Four Multivariate Manifold Simulation Settings

We explore geodesic learning using the following four simulations settings, as shown in Figure 1, each of which span a complementary and interesting case. In the linear case, where learning the geodesic should be relatively easy, Euclidean distance will completely recover the geodesics with no noise. It therefore sets an upper bound on performance. The helix setting is reminiscent of the typical “swiss jelly roll” setting popular in manifold learning, but the latent submanifold is one-dimensional embedded into a three-dimensional space. Here, Euclidean will perform poorly, but various manifold learning algorithms should perform well, as they are designed for this kind of scenario. The sphere case is interesting because unlike the helix, the true manifold is two-dimensional and could easily be extended to higher dimensions. Finally, the Gaussian mixture model can be particularly challenging for the manifold learning algorithms, which typically lack theoretical guarantees for graphs with disconnected components. Appendix C provides the mathematical details for the four different settings.

4.2 Choosing the Splitting Criteria and Robustness to Algorithm Parameters

For each of the above simulation settings, we sample a thousands points and calculate the geodesic precision using different geodesic random forest variants. Figure 2 shows an empirical comparison of the three different splitting criteria when used in an unsupervised random forest (URF) that chooses split points in the original feature space and in GF. The BIC approaches (red and blue) outperform two-means splitting criteria (green) in most cases. The solid and dashed lines show the relative performance of URF as compared to GF. GF uses sparse oblique splits, i.e., it splits on linear combinations of the original features. In most cases, GF outperforms URF. This is expected based on previous comparisons of sparse oblique splits to axis-aligned splits in supervised random forest [7, 47, 48]. In all cases, GF using Fast-BIC performs as well, or nearly as well, as the other options. Because it performs as well as other options and

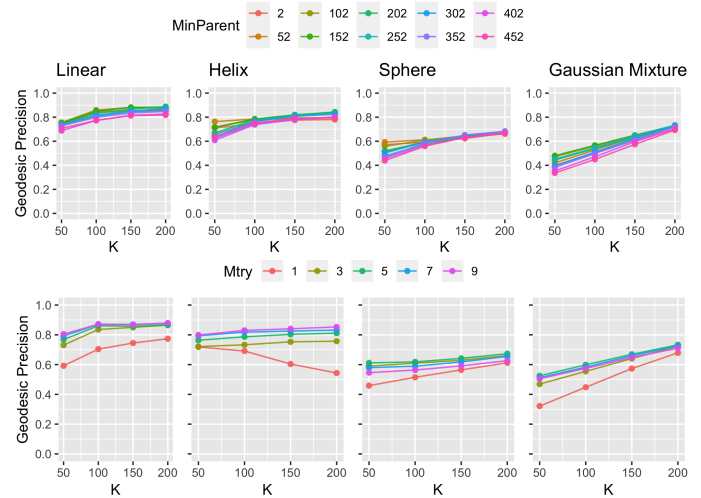


Figure 3: Top Geodesic precision versus k for different values of minparent (the smallest splittable node size). Mtry is set to be equal to the square root of the number of features. Bottom Geodesic precision versus k for different values of mtry (the number of features to test at each node). Minparent was set to be equal to 100. Geodesic precision is robust to large variations in these parameters

runs as fast as two-means, we elect to use GF+Fast-BIC (hereafter, simply GF) as our unsupervised decision forest splitting criteria.

In addition to the splitting criteria, each decision tree has two other important algorithm parameters. First, minparent, which sets the cardinality of the smallest node that might be split. Second, mtry, which is the number of features to test at each node. Figure 3 shows the geodesic precision for different values of minparent and mtry. Geodesic precision using GF is robust to hyperparameter changes, obviating the need for tuning hyperparameters via a grid search, which can be computationally intensive. For all future experiments, we set minparent to 100 and mtry to $\sqrt{d}$.

4.3 GF is Robust to Noise Dimensions

To see that GF is robust to high dimensional noise, Gaussian noise with varying dimensions d' are concatenated onto the simulated datasets. Specifically, for each data point $x_n \in \mathbb{R}^d$, generated noise $y_n \stackrel{iid}{\sim} \mathcal{N}(0, c\mathbb{I})$ where $y_n \in \mathbb{R}^{d'}$ is concatenated onto x_n , ($c = 70$ in the following experiments), and $\mathbb{I}$ is the $d' \times d'$ identity matrix. The new data points with noise are thus: $\tilde{x}_n = [x_n^\top | y_n^\top]^\top \in \mathbb{R}^{d+d'}$. Each algorithm’s proximity matrices are computed on the $\tilde{x}$ ’s and compared with geodesic distance matrices to obtain geodesic precision and recall.

Figure 4 shows the geodesic precision @ $k=50$ as a function of the number noise dimensions for Isomap, UMAP, random forests, Euclidean distance, FLANN, and GF. GF performs well even with the addition of high dimensional noise dimensions. The other algorithms achieve a higher geodesic recall than GF in the absence of noise dimensions, but degrade much more quickly than GF upon the addition of noise dimensions. FLANN and Euclidean distance

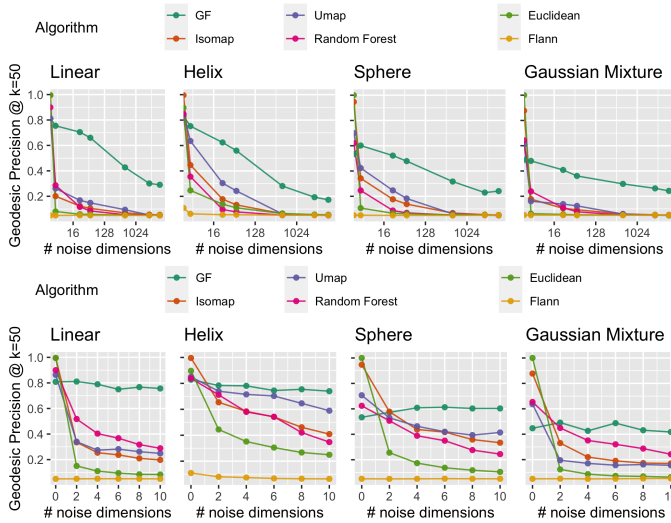


Figure 4: *Top* Geodesic precision at $k=50$ varying the noise dimensions from 2 to 10,000 with $N = 1000$ samples. Previous algorithms degrade to chance levels in all settings as the number of noise dimensions increases, GF maintains performance in all of the settings. *Bottom* Same as top, but each dimension is linearly rescaled to be between 0 and 1, and x-axis shows a smaller number of dimensions (from 0 to 10). Although rescaling greatly improves geodesic precision and recall for most algorithms, GF still achieves much larger geodesic precision in the presence of noise.

degrade the fastest, followed by Isomap and UMAP. This suggests that typical approximate nearest neighbor algorithms (which are approximating the distance in the ambient space) will perform poorly on recalling the desired items when the data live near a low-dimensional manifold. The top panel shows the geodesic precision curves for adding up to 10,000 dimensions. The performance of all the algorithms except GF degrades to chance levels in all four settings, whereas GF maintains a geodesic precision far above chance levels. The bottom panel shows geodesic precision after normalizing each of the dimensions (i.e., linearly rescaling each feature to be between 0 and 1). Other algorithms are sensitive to dimension rescaling and performance may improve as a result. However, GF consistently performs better, even without rescaling.

4.4 GF Estimates Geodesic Recall on Drosophila Connectome

The study of brain networks, or connectomics, is quickly emerging as an important source of real world data challenges [49]. Recently, the entire larval *Drosophila* mushroom body connectome—the learning and memory system of the fly—was estimated and released [20]. It was obtained via manual labeling and semi-automatic machine vision segmentation of serial section transmission electron microscopy. The 200 nodes of this connectome correspond to 200 distinct neurons in the mushroom body. There are roughly 75000 edges, defined as present between a pair of neurons whenever

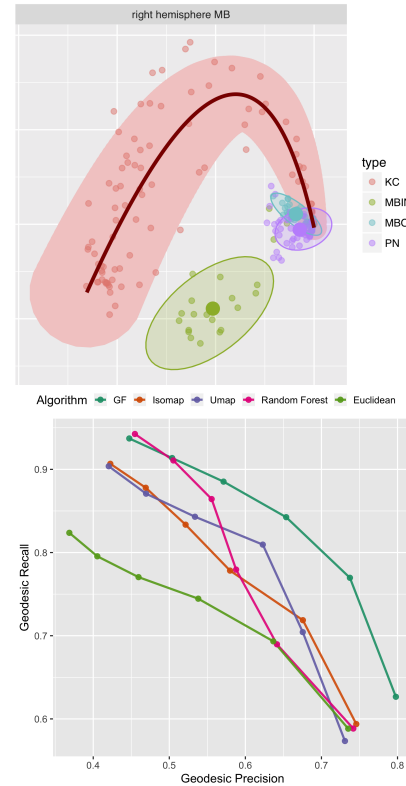


Figure 5: *Top* The right *Drosophila* connectome after adjacency spectral embedding into six-dimensional space just showing two of the dimensions, with colors corresponding to different cell types: Kenyon Cells (KC), Input Neurons (MBIN), Output Neurons (MBON), and Projection Neurons (PN). *Bottom* Geodesic precision versus geodesic recall for various algorithms using cell type as the true label. GF achieves a higher recall for essentially all precisions. The values of k for this experiment range from 50 to 250 with increments of 50

there exists at least one synapse between them. The edges connect vertices in four known classes of cells: kenyon cells, input neurons, output neurons, and projection neurons [35]. A semi-parametric analysis of the connectome, using adjacency spectral embedding [44], results in a six-dimensional latent representation of each node [35]. Because this connectome is directed, the first three dimensions correspond to “outgoing” latent features, whereas the next three correspond to “incoming” latent features. Figure 5 (top) shows two of the six dimensions. The bottom panel of Figure 5 shows the geodesic precision versus geodesic recall for various algorithms using cell type as the true label. GF achieves a higher recall at essentially all precision levels.

5 DISCUSSION

We present a geodesic distance learning method using Geodesic Forests (GF) with a novel splitting rule called Fast-BIC. We also introduce the geodesic precision and recall evaluation metrics. GF is

empirically robust to noise dimensions, as demonstrated by several different simulation settings, many different added noise dimensions, and the real-world *Drosophila* connectome. Geodesic learning is an essential statistical primitive for many subsequent inference tasks. For example, manifold learning, high-dimensional clustering, anomaly detection, and vertex nomination [52] all rely on geodesic learning. More generally, any *Learning to rank* (LTR) problem can be formulated as a geodesic learning problem. An LTR problem aims to provide an optimal ordering of a collection of items. Rather than computing an absolute score for each item, it computes a relative score between two or more items. This score is often a function of some distance between the two items. In LTR problems such as recommendation systems, the items to recommend often live in a high dimensional noisy space. In such a case, we hypothesize that the geodesic distance is an appropriate distance measure. Moreover, while we only considered unsupervised geodesic learning, the ideas presented here immediately lend themselves to supervised geodesic learning as well.

We did not explore any theoretical claims associated with the algorithms presented here. Indeed, we did not even evaluate whether any of these algorithms approximate the precise geodesic. Rather, our metric is concerned purely with getting the geodesic neighbors correct. However, prior work using random projections to learn low-dimensional manifolds [15] and for vector quantization [16] have theoretical guarantees associated with the intrinsic dimension of an assumed latent manifold. We posit that those guarantees could be transferred to this setting. Another potential direction would be to address the theoretical bounds that geodesic learning can provide with respect to Bayes optimal performance on both unsupervised and supervised learning problems. For example, 1-nearest neighbor provides tight bounds on Bayes optimal classification [5, 18]. Ideas presented in previous work on bounding Bayes performance using ranking algorithms could also extend to this setting.

GF is available as part of the open source package “SPORF” which includes a Python as well as an R package, available at <https://neurodata.io/sporf/>.

ACKNOWLEDGMENTS

The authors are grateful for the support by the D3M program of the Defense Advanced Research Projects Agency (DARPA), and DARPA’s Lifelong Learning Machines program through contract FA8650-18-2-7834.

REFERENCES

- [1] David Arthur and Sergei Vassilvitskii. 2007. k-means++: The advantages of careful seeding. In *Proceedings of the eighteenth annual ACM-SIAM symposium on Discrete algorithms*. dl.acm.org, 1027–1035. <https://dl.acm.org/citation.cfm?id=128338>
- [2] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2017. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. In *Similarity Search and Applications*. Springer International Publishing, 34–49. https://doi.org/10.1007/978-3-319-68474-1_3
- [3] Matej Balog, Balaji Lakshminarayanan, Zoubin Ghahramani, Daniel M Roy, and Yee Whye Teh. 2016. The Mondrian Kernel. (June 2016). [arXiv:stat.ML/1606.05241](https://arxiv.org/abs/1606.05241)
- [4] Mikhail Belkin and Partha Niyogi. 2002. Laplacian eigenmaps and spectral techniques for embedding and clustering. In *Advances in neural information processing systems*. 585–591.
- [5] GÅrard Biau, Luc Devroye, and GÅrbor Lugosi. 2008. Consistency of random forests and other averaging classifiers. *Journal of Machine Learning Research* 9, Sep (2008), 2015–2033.
- [6] Leo Breiman. 2000. *Some infinity theory for predictor ensembles*. Technical Report. Technical Report 579, Statistics Dept. UCB. <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.24.7078&rep=rep1&type=pdf>
- [7] Leo Breiman. 2001. Random forests. *Machine learning* 45, 1 (2001), 5–32.
- [8] Leo Breiman, Jerome Friedman, Charles J Stone, and R A Olshen. 1984. *Classification and Regression Trees (Wadsworth Statistics/Probability)* (1 edition ed.). Chapman and Hall/CRC. <https://www.amazon.com/Classification-Regression-Wadsworth-Statistics-Probability/dp/0412048418>
- [9] Rich Caruana, Nikos Karampatziakis, and Ainur Yessenalina. 2008. An empirical evaluation of supervised learning in high dimensions. In *Proceedings of the 25th international conference on Machine learning*. ACM, New York, New York, USA, 96–103. <https://doi.org/10.1145/1390156.1390169>
- [10] Rich Caruana and Alexandru Niculescu-Mizil. 2006. An Empirical Comparison of Supervised Learning Algorithms. In *Proceedings of the 23rd International Conference on Machine Learning (Pittsburgh, Pennsylvania, USA) (ICML '06)*. ACM, New York, NY, USA, 161–168. <https://doi.org/10.1145/1143844.1143865>
- [11] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (San Francisco, California, USA) (KDD '16)*. ACM, New York, NY, USA, 785–794. <https://doi.org/10.1145/2939672.2939785>
- [12] Ronald R Coifman and S Lafon. 2006. Diffusion maps. *Appl. Comput. Harmon. Anal.* 21, 1 (July 2006), 5–30.
- [13] A Criminisi and J Shotton. 2013. Manifold forests. In *Decision Forests for Computer Vision and Medical Image Analysis*. Springer, 79–93.
- [14] Sanjoy Dasgupta and Yoav Freund. 2008. Random projection trees and low dimensional manifolds.. In *STOC*, Vol. 8. Citeseer, 537–546.
- [15] Sanjoy Dasgupta and Yoav Freund. 2008. Random projection trees and low dimensional manifolds.. In *STOC*, Vol. 8. Citeseer, 537–546.
- [16] Sanjoy Dasgupta and Yoav Freund. 2008. Random projection trees for vector quantization. *IEEE Trans. Inf. Theory* 7 (May 2008), 3229–3242. [arXiv:stat.ML/0805.1390](https://arxiv.org/abs/0805.1390)
- [17] Alex Davies and Zoubin Ghahramani. 2014. The Random Forest Kernel and other kernels for big data from random partitions. (Feb. 2014). [arXiv:stat.ML/1402.4293](https://arxiv.org/abs/1402.4293)
- [18] Luc Devroye, Laszlo Györfi, and Gabor Lugosi. 1997. *A Probabilistic Theory of Pattern Recognition (Stochastic Modelling and Applied Probability)* (corrected edition ed.). Springer.
- [19] David L Donoho and Carrie Grimes. 2003. Hessian eigenmaps: locally linear embedding techniques for high-dimensional data. *Proc. Natl. Acad. Sci. U. S. A.* 100, 10 (May 2003), 5591–5596.
- [20] Katharina Eichler, Feng Li, Ashok Litwin-Kumar, Youngser Park, Ingrid Andrade, Casey M Schneider-Mizell, Timo Saumweber, Annina Huser, Claire Eschbach, Bertram Gerber, et al. 2017. The complete connectome of a learning and memory centre in an insect brain. *Nature* 548, 7666 (2017), 175.
- [21] Chris Fraley and Adrian E Raftery. 2002. Model-Based Clustering, Discriminant Analysis, and Density Estimation. *J. Amer. Statist. Assoc.* 97, 458 (June 2002), 611–631. <https://doi.org/10.1198/016214502760047131>
- [22] Y Freund and R E Schapire. 1997. A decision-theoretic generalization of on-line learning and an application to boosting. *J. Comput. System Sci.* (1997). <https://www.sciencedirect.com/science/article/pii/S002200009791504X>
- [23] Jerome H Friedman. 2001. Greedy function approximation: a gradient boosting machine. *Annals of statistics* (2001), 1189–1232.
- [24] Wayne A Fuller. 1987. *Measurement Error Models* (99 edition ed.). Wiley. https://smile.amazon.com/Measurement-Error-Models-Wayne-Fuller/dp/0471861871/ref=sr_1_2?keywords=Measurement+Error+Models&qid=1561237036&s=gateway&sr=8-2
- [25] David J Hand. 2016. *Measurement: A Very Short Introduction (Very Short Introductions)* (1 edition ed.). Oxford University Press. <https://www.amazon.com/Measurement-Very-Short-Introduction-Introductions/dp/0198779569>
- [26] J A Hartigan and M A Wong. 1979. Algorithm AS 136: A K-Means Clustering Algorithm. *Applied statistics* 28, 1 (1979), 100. <https://doi.org/10.2307/2346830>
- [27] John A Lee and Michel Verleysen. 2007. *Nonlinear Dimensionality Reduction* (2007 edition ed.). Springer Science & Business Media. <https://market.android.com/details?id=book-e4qkkQEACAAJ>
- [28] Andy Liaw and Matthew Wiener. 2002. Classification and Regression by random Forest. *R News* 2, 3 (2002), 18–22.
- [29] Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing Data using t-SNE. *J. Mach. Learn. Res.* 9, Nov (2008), 2579–2605. <http://www.jmlr.org/papers/v9/vandermaaten08a.html>
- [30] Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, Nov (2008), 2579–2605.
- [31] Leland McInnes and John Healy. 2018. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426* (2018).
- [32] Geoffrey McLachlan and Thriyambakam Krishnan. 2008. *The EM Algorithm and Extensions* (2 edition ed.). Wiley-Interscience. <https://www.amazon.com/EM-Algorithm-Extensions-Geoffrey-McLachlan/dp/0471201707>
- [33] Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. 2018. *Foundations of Machine Learning*. MIT Press. <https://market.android.com/details?id=book->

- dWB9DwAAQBAJ
- [34] Marius Muja and David G Lowe. 2014. Scalable nearest neighbor algorithms for high dimensional data. *IEEE Transactions on Pattern Analysis & Machine Intelligence* 11 (2014), 2227–2240.
 - [35] Carey E Priebe, Youngser Park, Minh Tang, Avanti Athreya, Vince Lyzinski, Joshua T Vogelstein, Yichen Qin, Ben Cocanougher, Katharina Eichler, Marta Zlatic, et al. 2017. Semiparametric spectral modeling of the *Drosophila* connectome. *arXiv preprint arXiv:1705.03297* (2017).
 - [36] Bernhard Schölkopf, Alexander Smola, and Klaus-Robert Müller. 1997. Kernel principal component analysis. In *International Conference on Artificial Neural Networks*. Springer, 583–588.
 - [37] Bernhard Schölkopf and Alexander J Smola. 2002. *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press.
 - [38] E Scornet. 2016. Random Forests and Kernel Methods. *IEEE Trans. Inf. Theory* 62, 3 (March 2016), 1485–1500. <http://dx.doi.org/10.1109/TIT.2016.2514489>
 - [39] Cen Cheng Shen and Joshua T Vogelstein. 2018. Decision Forests Induce Characteristic Kernels. (Nov. 2018). *arXiv:stat.ML/1812.00029* <http://arxiv.org/abs/1812.00029>
 - [40] Tao Shi and Steve Horvath. 2006. Unsupervised Learning With Random Forest Predictors. *J. Comput. Graph. Stat.* 15, 1 (March 2006), 118–138.
 - [41] Tao Shi and Steve Horvath. 2006. Unsupervised learning with random forest predictors. *Journal of Computational and Graphical Statistics* 15, 1 (2006), 118–138.
 - [42] Vin D Silva and Joshua B Tenenbaum. 2003. Global Versus Local Methods in Nonlinear Dimensionality Reduction. In *Advances in Neural Information Processing Systems 15*, S Becker, S Thrun, and K Obermayer (Eds.). MIT Press, 721–728. <http://papers.nips.cc/paper/2141-global-versus-local-methods-in-nonlinear-dimensionality-reduction.pdf>
 - [43] Charles J Stone. 1977. Consistent Nonparametric Regression. *Annals of statistics* 5, 4 (July 1977), 595–620. <https://doi.org/10.1214/aos/1176343886>
 - [44] Daniel L Sussman, Minh Tang, Donniell E Fishkind, and Carey E Priebe. 2012. A consistent adjacency spectral embedding for stochastic blockmodel graphs. *J. Amer. Statist. Assoc.* 107, 499 (2012), 1119–1128.
 - [45] Joshua B Tenenbaum, Vin De Silva, and John C Langford. 2000. A global geometric framework for nonlinear dimensionality reduction. *science* 290, 5500 (2000), 2319–2323.
 - [46] William C Thibault and Bruce F Naylor. 1987. Set operations on polyhedra using binary space partitioning trees. In *ACM SIGGRAPH computer graphics*, Vol. 21. ACM, 153–162.
 - [47] T Tomita, M Maggioni, and J Vogelstein. 2017. ROFLMAO: Robust Oblique Forests with Linear Matrix Operations. In *Proceedings of the 2017 SIAM International Conference on Data Mining*. Society for Industrial and Applied Mathematics, 498–506. <https://doi.org/10.1137/1.9781611974973.56>
 - [48] Tyler M Tomita, Mauro Maggioni, and Joshua T Vogelstein. 2015. Randomer forests. *arXiv preprint arXiv:1506.03410* (2015).
 - [49] Joshua T Vogelstein, Eric W Bridgeford, Benjamin D Pedigo, Jaewon Chung, Keith Levin, Brett Menseh, and Carey E Priebe. 2019. Connectal coding: discovering the structures linking cognitive phenotypes to individual histories. *Current opinion in neurobiology* 55 (2019), 199–212.
 - [50] Joe H Ward. 1963. Hierarchical Grouping to Optimize an Objective Function. *J. Amer. Statist. Assoc.* 58, 301 (March 1963), 236–244. <https://doi.org/10.1080/01621459.1963.10500845>
 - [51] Xindong Wu, Vipin Kumar, J Ross Quinlan, Joydeep Ghosh, Qiang Yang, Hiroshi Motoda, Geoffrey J McLachlan, Angus Ng, Bing Liu, Philip S Yu, Zhi-Hua Zhou, Michael Steinbach, David J Hand, and Dan Steinberg. 2007. Top 10 Algorithms in Data Mining. *Knowledge and information systems* 14, 1 (Dec. 2007), 1–37. <https://doi.org/10.1007/s10115-007-0114-2>
 - [52] Jordan Yoder, Li Chen, Henry Pao, Eric Bridgeford, Keith Levin, Donniell Fishkind, Carey Priebe, and Vince Lyzinski. 2018. Vertex nomination: The canonical sampling and the extended spectral nomination schemes. *arXiv preprint arXiv:1802.04960* (2018).

A SUPPLEMENTAL FIGURES

Additional experiments demonstrating GF’s ability to correctly estimate geodesic distances are provided in Figure 6. Here, we further explore the effect of low-level noise, similar to the experiments presented in Section 4. We vary the number of noise dimensions from 2 to 10, and compare GF to the same algorithms as in Section 4, but this time on unnormalized data. GF achieves high geodesic precision for all simulation settings, while the remaining algorithms degrade in performance significantly, even in this setting of low-level noise.

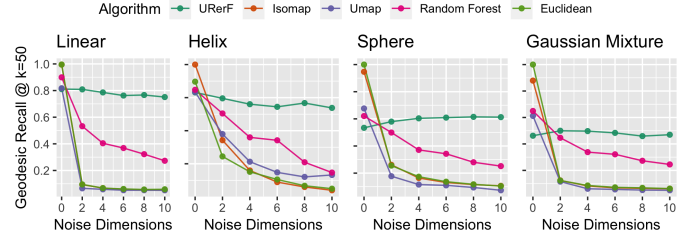


Figure 6: Geodesic precision at $k=50$ with varying noise dimension d' from 2 to 10, using $N = 1000$ samples, and $p = 3$ non-noisy dimensions. GF is robust to adding noise dimensions while the other algorithms deteriorate to chance in performance.

B ALGORITHMS

The pseudocode used to build each tree in a geodesic forest is given in Algorithm 1. The splitting methods used in this paper are described in algorithms 2 and 3.

Algorithm 1 Build a geodesic decision tree. Using sparse linear combinations of features for 1-dimensional projections, find the best splitting point among d of such projections.

```

1: procedure BUILDTREE( $X, d, \Theta$ )
2: Input:
3:  $X$  : a subset the of training data of dimension  $p$ 
4:  $d$  : dimensionality of the projected space
5:  $\Theta$  : set of split eligibility criteria
6: Output: a tree  $t$ 
7: if  $\Theta$  not satisfied then
8:   return LeafNode( $X$ ) ▷ Create a leaf node
9: else
10:   $A \leftarrow [a_1 \dots a_p] \sim f_A$  ▷ sample random  $p \times d$  matrix
11:   $\tilde{X} = A^T X$  ▷ random projection into new feature space
12:   $\min\_t^* \leftarrow \infty$ 
13:  for  $i \in \{1, \dots, d\}$  do ▷ test each projected dimension
14:     $\tilde{X}^{(i)} \leftarrow \tilde{X}[:, i]$ 
15:     $(\text{midpt}, t^*) = \text{ChooseSplit}(\tilde{X}^{(i)})$  ▷ Use either
16:    if  $(t^* < \min\_t^*)$  then ▷ store best splitting
17:       $\text{bestDim} = i$ 
18:       $\text{splitPoint} = \text{midpt}$ 
19:    end if
20:  end for
21:   $X_{\text{left}} = \{x \in X | x(\text{bestDim}) < \text{splitPoint}\}$ 
22:   $X_{\text{right}} = \{x \in X | x(\text{bestDim}) \geq \text{splitPoint}\}$ 
23:  Daughters.Left = BuildTree( $X_{\text{left}}, d, \Theta$ )
24:  Daughters.Right = BuildTree( $X_{\text{right}}, d, \Theta$ )
25:  return Daughters
26: end if
27: end procedure

```

Algorithm 2 Find the optimal split of one-dimensional data, in terms of the two-means objective.

```

1: procedure TwoMeans1D( $Z$ )
2: Input:  $Z \in \mathcal{R}^n$ : set of one-dimensional input values, one per
    $n$  data points
3: Output: Optimal two-means split point, splitPoint, between
   points in  $Z$  and the corresponding sum of squared distances to
   the two means, minVars
4:  $\hat{\mu}_1 \leftarrow \min(Z)$ 
5:  $C_1 \leftarrow \{\hat{\mu}_1\}$ 
6:  $C_2 \leftarrow Z \setminus C_1$ 
7:  $\hat{\mu}_2 \leftarrow \frac{1}{n_2} \sum_{z_i \in C_2} z_i$   $\triangleright$  mean of  $C_2$ 
8:  $\text{vars} \leftarrow \sum_{j=1}^2 \sum_{z_i \in C_j} (z_i - \hat{\mu}_j)^2$   $\triangleright$  sum of variances
9:  $\text{minVars} \leftarrow \text{vars}$ 
10: while  $C_2 \neq \emptyset$  do
11:    $z \leftarrow \min(C_2)$ 
12:    $C_1 \leftarrow C_1 \cup \{z\}$ 
13:    $C_2 \leftarrow C_2 \setminus \{z\}$ 
14:    $\hat{\mu}_1 \leftarrow \frac{1}{n_1} \sum_{z_i \in C_1} z_i$   $\triangleright$  mean of  $C_1$ 
15:    $\hat{\mu}_2 \leftarrow \frac{1}{n_2} \sum_{z_i \in C_2} z_i$   $\triangleright$  mean of  $C_2$ 
16:    $\text{vars} \leftarrow \sum_{j=1}^2 \sum_{z_i \in C_j} (z_i - \hat{\mu}_j)^2$ 
17:   if  $\text{vars} < \text{minVars}$  then
18:      $\text{minVars} \leftarrow \text{vars}$ 
19:      $\text{splitPoint} \leftarrow (\max(C_1) + \min(C_2))/2$   $\triangleright$  Midpoint
       between  $C_1$  and  $C_2$ 
20:   end if
21: end while
22: return (splitPoint, minVars)
23: end procedure

```

Algorithm 3 Fast-BIC1D: Find the optimal split, in terms of BIC score, of one-dimensional data with two classes.

```

1: procedure Fast-BIC1D( $Z$ )
2: Input:  $Z \in \mathcal{R}^n$ : data matrix containing  $n$  points in 1 dimension
3: Output: (splitPoint, minBIC): The midpoint and estimated BIC
   score that correspond to the best partition between the points
   in  $Z$ 
4:  $\hat{\mu}_1 \leftarrow \min(Z)$ 
5:  $C_1 \leftarrow \{\hat{\mu}_1\}$ 
6:  $C_2 \leftarrow Z \setminus C_1$ 
7:  $\hat{\mu}_2 \leftarrow \frac{1}{|C_2|} \sum_{z_i \in C_2} z_i$   $\triangleright$  mean of  $C_2$ 
8:  $\text{BIC\_curr} \leftarrow \sum_{j=1}^2 \sum_{z_n \in C_j} (z_n - \mu_j)^2$ 
9:  $\text{minBIC\_curr} \leftarrow \text{dists\_sq}$ 
10: while  $C_2 \neq \emptyset$  do
11:    $z \leftarrow \min(C_2)$ 
12:    $C_1 \leftarrow C_1 \cup \{z\}$ 
13:    $C_2 \leftarrow C_2 \setminus \{z\}$ 
14:   for  $j = 1, 2$  do
15:      $n_j = |C_j|$ 
16:      $\hat{w}_j = n_j/n$ 
17:      $\hat{\mu}_j \leftarrow \frac{1}{n_j} \sum_{z_i \in C_j} z_i$   $\triangleright$  mean of  $C_j$ 
18:      $\hat{\sigma}_j^2 \leftarrow \frac{1}{n_j} \sum_{z_i \in C_j} (z_i - \hat{\mu}_j)^2$   $\triangleright$  variance of  $C_j$ 
19:   end for
20:    $\hat{\sigma}_{\text{comb}}^2 \leftarrow \frac{1}{n} \sum_{j=1}^2 \sum_{z_i \in C_j} (z_i - \hat{\mu}_j)^2$ 
21:    $\text{BIC\_diff\_var} \leftarrow -2(n_1 \log \hat{w}_1 - \frac{n_1}{2} \log 2\pi \hat{\sigma}_1^2 - n_2 \log \hat{w}_2 +$ 
      $\frac{n_2}{2} \log 2\pi \hat{\sigma}_2^2)$ 
22:    $\text{BIC\_same\_var} \leftarrow -2(n_1 \log \hat{w}_1 - \frac{n_1}{2} \log 2\pi \hat{\sigma}_{\text{comb}}^2 -$ 
      $n_2 \log \hat{w}_2 + \frac{n_2}{2} \log 2\pi \hat{\sigma}_{\text{comb}}^2)$ 
23:    $\text{BIC\_curr} \leftarrow \min(\text{BIC\_same\_var}, \text{BIC\_diff\_var})$ 
24:   if  $\text{BIC\_curr} < \text{minBIC}$  then
25:      $\text{minBIC} \leftarrow \text{BIC\_curr}$ 
26:      $\text{splitPoint} \leftarrow (\max(C_1) + \min(C_2))/2$   $\triangleright$  Midpoint
       between  $C_1$  and  $C_2$ 
27:   end if
28: end while
29: return (splitPoint, minBIC)
30: end procedure

```

C SIMULATION SETTINGS

Here we detail how each synthetic data set used in numerical experiments was generated. These data sets were chosen to capture a range of interesting cases that can illuminate the challenges of geodesic distance estimation in noisy settings.

- **Linear:** each point x is parameterized by $p = (4t, 6t, 9t)$, with $t \in (0, 1)$ where t is sampled from a grid with equal spacing.
- **Helix:** each point x is parameterized by $p = (t \cos(t), t \sin(t), t)$, with $t \in (2\pi, 9\pi)$ on an equally spaced grid.
- **Sphere:** each point x is parameterized by $p = (r \cos(u) \sin(v), r \sin(u) \sin(v), r \cos(v))$, with $u \in (0, 2\pi)$, $v \in (0, \pi)$ and $r = 9$ where u, v are sampled from a grid with equal spacing.

- **Gaussian Mixture:** each point x is drawn from a mixture of Gaussian distributions: $\sum_{j=1}^3 \hat{w}_j \mathcal{N}(\mu_j, \Sigma)$, with $(\pi_1, \pi_2, \pi_3) = (0.3, 0.3, 0.4)$, $(\hat{\mu}_1, \hat{\mu}_2, \hat{\mu}_3) = \left(\begin{bmatrix} -3 \\ -3 \\ -3 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 3 \\ 3 \\ 3 \end{bmatrix} \right)$ and $\Sigma = \mathbb{I}$ is the identity matrix.

D FAST-BIC1D DERIVATION

The BIC test is an alternative to the two means split criteria. We rank potential splits on the BIC score obtained assuming a model with a mixture of two Gaussians; the elements along a given dimension to the left of the cut point belong to one Gaussian and the elements to the right belong to another Gaussian (the elements are in sorted order). For example, if the elements are $[1, 3, 4, 6]$ along a dimension, possible splits are $[[1], [3, 4, 6]]$, $[[1, 3], [4, 6]]$, $[[1, 3, 4], [6]]$. In the case $[[1, 3], [4, 6]]$, we assume the model to comprise of two Gaussians, where $[1, 3]$ are sampled from the first and $[4, 6]$ are sampled from the second. We choose the split that results in the lowest BIC score. We then repeat the process for all dimensions and choose the dimension that gives the split resulting in the lowest BIC score. We use the following notation in the below derivation:

- Z : the set of n points in one-dimension
- s : the current split point
- n_j : Number of elements in cluster j
- w_j : probability of choosing cluster j , $w_j = \frac{n_j}{n}$
- μ_j, σ_j^2 : mean and variance of cluster j

Computing the log likelihood is slightly different here than for the regular GMM. This is because we already know which element belongs in which cluster, so we can compute the log likelihood directly, without the EM step.

$$\begin{aligned} \log \ell(Z) &= \log \ell(\{z_n\}_{n < s}) + \log \ell(\{z_n\}_{n > s}) \\ \log \prod_n P(x_n; \mu, \sigma^2, w) &= \sum_{n < s} [\log w_1 + \log \mathcal{N}(x_n; \mu_1, \sigma_1^2 I)] + \\ &\sum_{n > s} [\log w_2 + \log \mathcal{N}(x_n; \mu_2, \sigma_2^2 I)] \end{aligned} \quad (7)$$

Substituting the expression for w_1 and w_2 and expanding the log Gaussian, we get the following:

$$\begin{aligned} \log \prod_n P(x_n; \mu, \sigma^2, w) &= n_1 \log w_1 - \frac{n_1}{2} \log 2\pi \sigma_1^2 - \sum_{n_1} \frac{\|x_{n_1} - \mu_1\|^2}{2\sigma_1^2} + \\ &n_2 \log w_2 - \frac{n_2}{2} \log 2\pi \sigma_2^2 - \sum_{n_2} \frac{\|x_{n_2} - \mu_2\|^2}{2\sigma_2^2} \end{aligned} \quad (8)$$

The parameters $w_1, w_2, \mu_1, \mu_2, \sigma_1$ and σ_2 are unknown. We use the maximum likelihood estimates as a plug-in for each:

$$\begin{aligned} \hat{w}_1 &= n_1 / N, \\ \hat{\mu}_1 &= \frac{1}{n_1} \sum_{n < s} x_n, \\ \hat{\sigma}_1^2 &= \frac{1}{n_1} \sum_{n < s} \|x_n - \hat{\mu}_1\|^2, \end{aligned}$$

with the parameters for $j = 2$ defined equivalently. Thus the right hand side of Equation (8) can be expressed as

$$n_1 \log w_1 - \frac{n_1}{2} \log 2\pi \hat{\sigma}_1^2 - \frac{n_1}{2} + n_2 \log w_2 - \frac{n_2}{2} \log 2\pi \hat{\sigma}_2^2 - \frac{n_2}{2} \quad (9)$$

In the BIC formula, we compute the negative log likelihood. - $\log \prod_n P(x_n; \hat{\mu}, \hat{\sigma}^2, \hat{w}) = -n_1 \log \hat{w}_1 + \frac{n_1}{2} \log 2\pi \hat{\sigma}_1^2 + \frac{n_1}{2} - n_2 \log \hat{w}_2 + \frac{n_2}{2} \log 2\pi \hat{\sigma}_2^2 + \frac{n_2}{2}$.